Lab Assignment Solutions

L10 — Spring Boot REST API with JPA/ORM

Q10.1 [5 marks] Steps to set up Spring Boot + REST + JPA and role of embedded Tomcat

Setting up a Spring Boot project that exposes a REST API backed by JPA/Hibernate ORM involves the following steps:

- 1. Generate the project skeleton from Spring Initializr (start.spring.io) selecting Maven, Java, and the latest stable Spring Boot version.
- 2. Add the required Maven dependencies to pom.xml: spring-boot-starter-web (for REST endpoints and the embedded servlet container), spring-boot-starter-data-jpa (for ORM/Hibernate + Spring Data repositories), and the MySQL connector (mysql-connector-j) as the JDBC driver.
- 3. Configure the datasource in src/main/resources/application.properties — DB URL, username, password, and Hibernate settings such as spring.jpa.hibernate.ddl-auto=update.
- 4. Create the entity classes (annotated with @Entity) that map Java objects to database tables.
- 5. Create repository interfaces that extend JpaRepository to get CRUD operations without writing SQL.
- 6. Create @RestController classes that use the repositories to expose HTTP endpoints (GET, POST, PUT, DELETE).
- 7. Run the application via the main class annotated with @SpringBootApplication (or mvn spring-boot:run); Spring Boot auto-configures everything and starts the app.

pom.xml — key dependencies

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Role of the embedded Tomcat server:

- spring-boot-starter-web pulls in an embedded Tomcat server bundled inside the application's own JAR, so there is no need to install or configure a standalone Tomcat/Servlet container separately.
- It starts automatically when the Spring Boot application runs (default port 8080), turning the app into a self-contained, runnable executable ('java -jar app.jar').

- It listens for incoming HTTP requests and dispatches them to the Spring MVC DispatcherServlet, which routes them to the correct @RestController method — this is what makes the REST endpoints reachable over HTTP.
- Because the server is embedded, deployment is simplified (no external app-server, no WAR files needed) and each microservice can run independently with its own container instance.

Q10.2 [8 marks] JPA Student entity (id, name, cgpa) and StudentRepository

Student.java

```
package com.example.studentapi.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private Double cgpa;

    public Student() {
    }

    public Student(String name, Double cgpa) {
        this.name = name;
        this.cgpa = cgpa;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Double getCgpa() {
        return cgpa;
    }
```

```
    public void setCgpa(Double cgpa) {
        this.cgpa = cgpa;
    }
}
```

StudentRepository.java

```
package com.example.studentapi.repository;

import com.example.studentapi.model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
    // JpaRepository already provides:
    // save(), findAll(), findById(), deleteById(), count(), etc.
}
```

Extending JpaRepository<Student, Long> gives full CRUD functionality for free (Student = entity type, Long = type of the primary key), without writing any SQL or DAO boilerplate.

Q10.3 [7 marks] @RestController with GET /students and POST /students

StudentController.java

```
package com.example.studentapi.controller;

import com.example.studentapi.model.Student;
import com.example.studentapi.repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentRepository studentRepository;

    // GET /students -> list all students
    @GetMapping
    public ResponseEntity<List<Student>> getAllStudents() {
        return ResponseEntity.ok(studentRepository.findAll());
    }

    // POST /students -> add a new student
    @PostMapping
    public ResponseEntity<Student> addStudent(@RequestBody Student student) {
        Student saved = studentRepository.save(student);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

Sample output

Request: POST /students with body:

```
{
  "name": "Rahim Uddin",
  "cgpa": 3.75
}
```

Response (201 Created):

```
{
  "id": 1,
  "name": "Rahim Uddin",
  "cgpa": 3.75
}
```

Request: GET /students -> Response (200 OK):

```
[
  { "id": 1, "name": "Rahim Uddin",  "cgpa": 3.75 },
  { "id": 2, "name": "Karim Hossain","cgpa": 3.42 }
]
```

L11 — Servlet CRUD — District Quiz Game

Q11.1 [5 marks] DB schema for Crops / Geography / Academic Institutions quiz + Player/Score table

The quiz needs two tables: one to store the question bank (covering the three categories — crops, geography, and academic institutions of the district) and one to record each player's result.

Table: question

Column	Type	Notes
question_id	INT, PK, AUTO_INCREMENT	Unique identifier
category	VARCHAR(20)	'CROP', 'GEOGRAPHY', or 'INSTITUTION'
question_text	VARCHAR(255)	The MCQ prompt
option_a	VARCHAR(100)	Choice A
option_b	VARCHAR(100)	Choice B
option_c	VARCHAR(100)	Choice C
option_d	VARCHAR(100)	Choice D
correct_option	CHAR(1)	'A' | 'B' | 'C' | 'D'

Table: player_score

Column	Type	Notes
score_id	INT, PK, AUTO_INCREMENT	Unique identifier
player_name	VARCHAR(50)	Name entered by the player
final_score	INT	Total correct answers
played_on	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	When the game was played

Design justification:

- question and player_score are kept as two separate tables because they model two different real-world entities (static content vs. dynamic gameplay results) — this avoids data duplication and keeps updates to the question bank independent of player history.
- A category column (rather than three separate tables) lets one query fetch a random mix of crop, geography and institution questions, and makes it trivial to add new categories later without changing the schema.
- player_score has no foreign key to question because it stores only the aggregate result of a session, not per-question answers, which matches the requirement to 'save player name + final score'; the design can be extended with an answer_log(score_id, question_id, is_correct) table if per-question analytics are ever needed.
- played_on is auto-populated so leaderboards ('today's top scores') can be built without extra application logic.

Q11.2 [8 marks] Servlet method: save player name + final score into PlayerScore via JDBC/PreparedStatement

```
import java.sql.*;
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.IOException;

public class SaveScoreServlet extends HttpServlet {

    private static final String DB_URL  = "jdbc:mysql://localhost:3306/quizdb";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "root";

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String playerName = request.getParameter("playerName");
        int finalScore = Integer.parseInt(request.getParameter("finalScore"));

        String sql = "INSERT INTO player_score (player_name, final_score) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {

            ps.setString(1, playerName);
            ps.setInt(2, finalScore);

            int rows = ps.executeUpdate();
```

```
            response.setContentType("text/plain");
            if (rows > 0) {
                response.getWriter().println("Score saved: " + playerName + " -> " +
finalScore);
            } else {
                response.getWriter().println("Failed to save score.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
            response.getWriter().println("DB error: " + e.getMessage());
        }
    }
}
```

Using a PreparedStatement with ? placeholders (rather than concatenating strings) precompiles the SQL and binds parameters safely, which prevents SQL injection through the playerName field.

Sample output

POST request: playerName=Anika+Rahman&finalScore=3

```
Score saved: Anika Rahman -> 3

-- Resulting row in player_score --
score_id | player_name  | final_score | played_on
   4     | Anika Rahman |      3      | 2026-08-18 01:04:23
```

Q11.3 [7 marks] Quiz logic: 3+ MCQs (crop, geography, institution) as objects; present, check, increment score

MCQ.java — model class

```
public class MCQ {
    private String category;
    private String questionText;
    private String optionA, optionB, optionC, optionD;
    private char correctOption;

    public MCQ(String category, String questionText,
               String a, String b, String c, String d, char correctOption) {
        this.category = category;
        this.questionText = questionText;
        this.optionA = a; this.optionB = b; this.optionC = c; this.optionD = d;
        this.correctOption = correctOption;
    }

    public boolean isCorrect(char chosenOption) {
        return Character.toUpperCase(chosenOption) == correctOption;
    }

    public String getCategory()     { return category; }
    public String getQuestionText() { return questionText; }
    public String getOptionA() { return optionA; }
    public String getOptionB() { return optionB; }
    public String getOptionC() { return optionC; }
    public String getOptionD() { return optionD; }
}
```

QuizServlet.java — present, check answer, increment score

```
import java.util.*;
import javax.servlet.http.*;
import javax.servlet.*;
import java.io.IOException;

public class QuizServlet extends HttpServlet {

    private List<MCQ> loadQuestions() {
        List<MCQ> questions = new ArrayList<>();
        questions.add(new MCQ("CROP",
            "Which crop is Tangail district traditionally known for?",
            "Jute", "Tea", "Rubber", "Coffee", 'A'));
        questions.add(new MCQ("GEOGRAPHY",
            "Tangail district lies in which division of Bangladesh?",
            "Rajshahi", "Dhaka", "Khulna", "Sylhet", 'B'));
        questions.add(new MCQ("INSTITUTION",
            "Which is a well-known academic institution in Tangail?",
            "Mawlana Bhashani Science & Technology University",
            "Chittagong University", "Khulna University", "Jahangirnagar University",
'A'));
        return questions;
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        List<MCQ> questions = loadQuestions();
        int score = 0;

        response.setContentType("text/plain");
        var out = response.getWriter();

        for (int i = 0; i < questions.size(); i++) {
            MCQ q = questions.get(i);

            // Present the question
            out.println("Q" + (i + 1) + " [" + q.getCategory() + "]: " +
q.getQuestionText());
            out.println("  A) " + q.getOptionA());
            out.println("  B) " + q.getOptionB());
            out.println("  C) " + q.getOptionC());
            out.println("  D) " + q.getOptionD());

            // Answer submitted from the quiz form, e.g. ans1, ans2, ans3
            String answerParam = request.getParameter("ans" + (i + 1));
            char chosen = (answerParam != null && !answerParam.isEmpty())
                    ? answerParam.trim().charAt(0) : ' ';

            // Check answer and increment score
            if (q.isCorrect(chosen)) {
                score++;
                out.println("  -> Correct!");
            } else {
                out.println("  -> Wrong. Correct answer: " + q.correctOptionDisplay());
            }
            out.println();
        }

        out.println("Final Score: " + score + " / " + questions.size());
```

```
        // Persist the result (reuses the PreparedStatement pattern from 11.2)
        // new ScoreDao().save(request.getParameter("playerName"), score);
    }
}
```

Note: correctOptionDisplay() is a small convenience getter that returns the correctOption field for display in the 'Wrong' message; add it to MCQ.java alongside the other getters if you compile this directly.

Sample output

```
Q1 [CROP]: Which crop is Tangail district traditionally known for?
  A) Jute
  B) Tea
  C) Rubber
  D) Coffee
  -> Correct!

Q2 [GEOGRAPHY]: Tangail district lies in which division of Bangladesh?
  A) Rajshahi
  B) Dhaka
  C) Khulna
  D) Sylhet
  -> Correct!

Q3 [INSTITUTION]: Which is a well-known academic institution in Tangail?
  A) Mawlana Bhashani Science & Technology University
  B) Chittagong University
  C) Khulna University
  D) Jahangirnagar University
  -> Wrong. Correct answer: A

Final Score: 2 / 3
```

L12 — GoF Design Patterns

Q12.1 [6 marks] 5 GoF Creational patterns with one-line purpose each

Pattern	Purpose
Singleton	Ensures a class has only one instance and provides a single global point of access to it.
Factory Method	Defines an interface for creating an object but lets subclasses decide which concrete class to instantiate.
Abstract Factory	Provides an interface for creating families of related objects without specifying their concrete classes.
Builder	Separates the construction of a complex object from its representation, building it step by step.

Prototype	Creates new objects by copying (cloning) an existing instance rather than instantiating from scratch.

Q12.2 [7 marks] 7 GoF Structural patterns with one-line purpose each

Pattern	Purpose
Adapter	Converts the interface of a class into another interface clients expect, letting incompatible classes work together.
Bridge	Decouples an abstraction from its implementation so the two can vary independently.
Composite	Composes objects into tree structures to represent part-whole hierarchies, treating individual and composite objects uniformly.
Decorator	Attaches additional responsibilities to an object dynamically, as a flexible alternative to subclassing.
Facade	Provides a simplified, unified interface to a set of interfaces in a complex subsystem.
Flyweight	Uses sharing to support large numbers of fine-grained objects efficiently, minimizing memory use.
Proxy	Provides a surrogate or placeholder for another object to control access to it.

Q12.3 [7 marks] Implement Singleton (thread-safe) and Adapter with short, complete Java code examples

Singleton (thread-safe, double-checked locking)

```
public class DatabaseConnection {

    // volatile ensures visibility across threads during double-checked locking
    private static volatile DatabaseConnection instance;

    private DatabaseConnection() {
        // private constructor prevents external instantiation
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {                      // 1st check (no locking)
            synchronized (DatabaseConnection.class) {
                if (instance == null) {              // 2nd check (with locking)
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }

    public void connect() {
```

```
        System.out.println("Connected via instance: " + this.hashCode());
    }
}

// Usage
public class SingletonDemo {
    public static void main(String[] args) {
        DatabaseConnection c1 = DatabaseConnection.getInstance();
        DatabaseConnection c2 = DatabaseConnection.getInstance();
        c1.connect();
        c2.connect();
        System.out.println("Same instance? " + (c1 == c2));
    }
}
```

Sample output

```
Connected via instance: 366712642
Connected via instance: 366712642
Same instance? true
```

Adapter

```
// Target interface the client expects
interface MediaPlayer {
    void play(String fileName);
}

// Adaptee: an existing, incompatible class
class AdvancedPlayer {
    void playMp4(String fileName) {
        System.out.println("Playing MP4 file: " + fileName);
    }
}

// Adapter: makes AdvancedPlayer compatible with MediaPlayer
class MediaAdapter implements MediaPlayer {
    private AdvancedPlayer advancedPlayer = new AdvancedPlayer();

    @Override
    public void play(String fileName) {
        advancedPlayer.playMp4(fileName);
    }
}

// Client
public class AdapterDemo {
    public static void main(String[] args) {
        MediaPlayer player = new MediaAdapter();
        player.play("lecture_notes.mp4");
    }
}
```

Sample output

```
Playing MP4 file: lecture_notes.mp4
```